

Using TelecomRAG for analysis of 5G MAC layer logs

Final Presentation

Dhanraj Shrivastav (CS24MTECH11004)

Project Recap (Brief)

- Problem statement
 - Briefly restate the problem you have chosen (1-2 lines)
 - Build RAG pipeline to perform detailed analysis of MAC layer logs using an LLM
 - Mitigate the prompt dependency problem of LLM-5GMAC.
- Motivation
 - Why this matters
 - Closes the skill gap between novice and experienced telecom operators.
- Key Objectives
 - List out main objectives you proposed.
 - (continued on next slide)

Project Recap (Brief) (continued)

- Key Objectives

- List out main objectives you proposed.
 - Replicate prompt dependency finding of LLM5G-MAC.
 - Propose new scenarios causing MAC layer faults and collect relevant logs using OAI/ns3.
 - Evaluate whether LLMs are effective at providing detailed analyses of such faults
 - Evaluate whether RAG assists in eliminating prompt dependency
- Additional objectives that emerged:
 - Explore alternative data-driven approaches to mitigate prompt dependency and close the operator skill gap.
 - Compare and contrast the different approaches and assess the efficacy of each one.
(Comparative study)

Related Work / Positioning

- List 2–3 closest works/systems
 - LLM-5GMAC: <https://dl.acm.org/doi/10.1145/3737900.3770164>
 - TelecomRAG: <https://dl.acm.org/doi/10.1145/3711992.3711996>
- What they do?
 - LLM-5GMAC: highlights prompt dependency problem.
 - TelecomRAG: QA for 5G user questions using RAG on 3GPP specs.
- What gap your project addresses or reproduces?
 - Mitigate prompt dependency.
 - Examines various different methods to do so.

System / Threat Model

- System ~~or Threat~~ Model
 - Log generation source: USRP B210, ns-3 scenario
 - Event extraction module: extracts top-5 key events/anomalies from logs (done using multiple methods)
 - Knowledge base: 3GPP TS 38.321 (MAC protocol spec), [mac-usage.md](#) from OAI
 - Retrieval module: RAG + retrieval query building (concat, LLM translate)
 - Large language models: gemma4:e4b for analyzing logs, llama-3.3-70b-versatile for other tasks (log template classification, log translation, etc.)

System / Threat Model (continued)

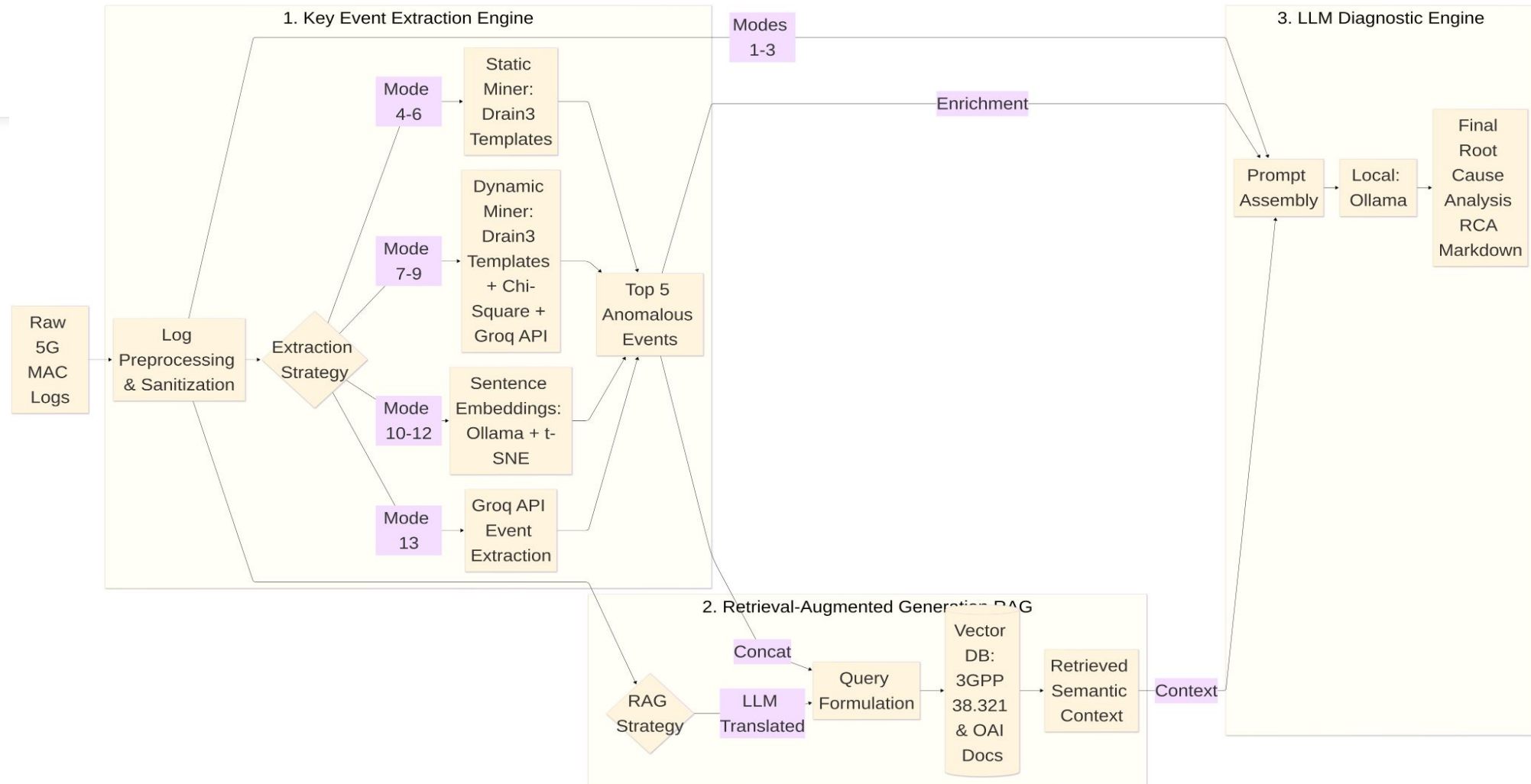
- Key Assumptions

- List your assumptions (1-2 lines)
 - The logs are such that they actually do contain identifiable key events. Subtle metric changes will be difficult to identify and consider as key events.
 - For the sentence embedding approach, assumption is that the key events are rarer than normal events and tend to form smaller clusters.

System Architecture/Methodology (1-2 slides)

- Proposed Solution Approach
 - Briefly show your high-level design diagram or solution approach
 - Highlight the core modules
- Data/control flow
- Highlight your contributions!

High Level Design Diagram + Data/Control Flow



High Level Design Diagram + Data/Control Flow (Highlight your contribution)

- Collecting Raw MAC logs
- Log Preprocessing and Sanitization
- Extraction Strategies: All four.
- RAG: both concat and llm_translate.
- Generating all relevant artifacts from the pipeline
- All parts in the diagram on previous slide.

Key Algorithm-1 (Static Miner)

```
// 1. Static Miner: Heuristic Scoring Routine
FUNCTION Score_By_Heuristics(template_freq, ordered_templates,
log_lines):
    total_lines = Length(log_lines)
    first_occurrence = EMPTY_MAP

    // Step 1: Temporal Mapping
    // Identify the first line index where each template appears.
    // Logic: Errors appearing earlier in the sequence are more
    // likely to be the root cause.
    FOR index, template IN ordered_templates:
        IF template NOT IN first_occurrence:
            first_occurrence[template] = index

    // Step 2: Vocabulary Initialization
    // Define and stem hardcoded telecom/network error keywords.
    significant_terms = ["fail", "invalid", "loss", "drop",
"timeout", "exceed", "mismatch", "unknown", "abort", "reject",
"release", "disconnect", "retx", "nack", "corrupt", "failure"]
    significant_stems = Stem_Words(significant_terms)

    scores = EMPTY_MAP
    // Step 3: Composite Scoring Matrix
    FOR template, count IN template_freq:

        // A. Rarity Weighting (Range: ~0.0 to 1.0)
        // Highly frequent logs (noise) approach 0; rare anomalies approach
        // 1.
        rarity_score = 1 - (count / total_lines)
```

```
// B. Semantic Significance Weighting
// Tokenize the template, stem the words, and count matches against the
// hardcoded error dictionary.
template_stems = Stem_Words(Tokenize(template))
significance_score = Count_Overlaps(template_stems, significant_stems)

// C. Temporal Weighting (Range: 0.0 to 1.0)
// Templates appearing at the top of the log approach 1; end of the log
// approach 0.
position = first_occurrence[template]
temporal_score = 1 - (position / total_lines)

// D. Final Formula Calculation
// Adds 1 to semantic and temporal scores to prevent zero-multiplication
// nullification.
scores[template] = rarity_score * (significance_score + 1) * (1 +
temporal_score)

// Step 4: Output
RETURN Sort_Descending(scores)
```

Key Algorithm-2 (Dynamic Miner)

Instead of hardcoding the significant word list. This algorithm simply learns this list from the corpus. This allows the system to generalize to new unseen anomalies.

First, an LLM (llama-3.3-40b-versatile) is used to classify Drain3 templates into NORMAL or ANOMALOUS using few-shot prompting.

This gives us two classes of templates. Extract all words from all the templates. Now, a chi-squared test is applied to find the words that skew towards the ANOMALOUS class. This list of words is the learned significant word list.

For example, for invalid TA offset related causal chain, the learned significant words are:

[timing,ignore,offset,ulsch,reported,advance,unexpected,detected,proc,pid,header,invalid,warning,unknown,dtx,infeasible,harq,residual,length]

For harq exhaustion because of weak link the learned list is:

[sched,stall,mcs,feedback,ri,nack,b,cqi,nr,scheduling,tbs,wb,harq,exhausted]

The rest of the scoring is done as in static mining as:

$rarity_score * (1 + significance_score) * label_weight$

$label_weight = 10$ if $label=ANOMALOUS$, 0.01 if $label=NORMAL$ (discards normal templates)

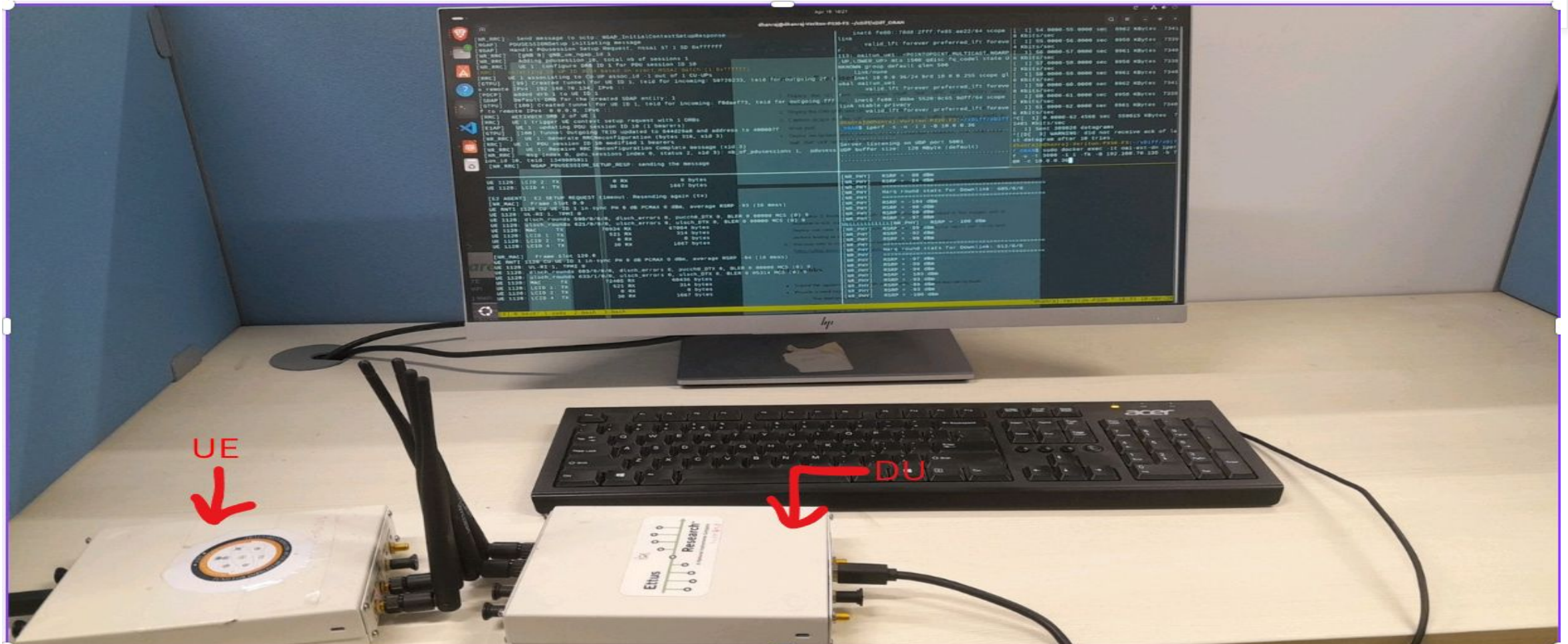
Key Algorithms-3 and 4

1. Agglomerative Clustering (Embeddings): Sentences are converted into dense vector representations. Then agglomerative clustering with $k=15$ clusters is done. The five smallest clusters and the majority log lines in them are taken as the key events.
2. LLM as event extractor: The cleaned up logs are given to a large LLM *llama-3.3-70b-versatile*. It is prompted to extract 5 key events from these logs.

Implementation Details (1-2 slides)

- Hardware Setup (if applicable):
 - USRP B210 for initial scenario.
 - (Already shown in mid-term presentation; picture included on next slide.)
- Software & Tools:
 - Ollama, LangChain, ChromaDB, Drain3, Scikit-learn, Plotly, Groq API, Docling, OAI, ns-3
- Implementation:
 - (Walk-through the key code snippets, configurations, simulation/emulation parameters, topology, or scripts you have written by showing your GitHub repo):
<https://github.com/dhanraj-s/TWiN-IITH-5GMACTLogAnalysis>
 - What you built vs reused?
 - Built: all parts of pipeline shown in slide 8 (total of 13 modes); Reused: Common libraries
 - Key challenges faced and how did you address them?
 - Limiting API credits consumed for call to Groq API (used caching of LLM outputs)
 - template_classifications, translated retrieval query.

Picture of setup for initial scenario



Experimental Setup

- Setup
 - Initial scenarios: usrp disconnect, ns-3 harq exhaustion
- Reproducibility details
 - All packages/modules used present in *requirements.txt*
 - NVIDIA A6000 GPU used for gemma4:e4b, qwen-3-embedding:4b, docling pdf to markdown conversion
- Metrics used for Analysis
 - Qualitative Analysis Done (see next section)
- Baseline(s) for Comparison
 - LLM with general prompt + no RAG (Mode 1)
 - LLM with general prompt + Concat RAG (Mode 2)
 - LLM with general prompt + LLM translation RAG (Mode 3)

Different Modes/Methods studied

- **Mode 1 (Pure Baseline)**
- **Mode 2 (Baseline + Concat RAG)**
- **Mode 3 (Baseline + LLM Translated RAG)**
- **Mode 4 (Static + No RAG)**
- **Mode 5 (Static + Concat RAG)**
- **Mode 6 (Static + LLM Translated RAG)**
- **Mode 7 (Dynamic + No RAG)**
- **Mode 8 (Dynamic + Concat RAG)**
- **Mode 9 (Dynamic + LLM Translated RAG)**
- **Mode 10 (Embeddings + No RAG)**
- **Mode 11 (Embeddings + Concat RAG)**
- **Mode 12 (Embeddings + LLM Translated RAG)**
- **Mode 13 (Groq Extraction + No RAG)**

Results & Analysis - 1

- Observation #1: Generic prompt does not reveal causal chain, even with RAG (both forms concat+LLM translation)
- Evidence to support: None of the general prompt outputs (modes 1,2,3 with concat rag, llm_translation rag) gave any meaningful causal chain analysis.

(see artifacts)

- On the right, is mode3 analysis (irrelevant parts cut off)
- Explanation: Simply retrieving more information about 5G MAC specs does not help the LLM reason deeply about root causes

```
This analysis focuses on the provided MAC layer logs, which contain detailed metrics regarding signal strength, data throughput, error rates, and synchronization status.

## 📶 Wireless Link Quality Analysis

Overall, the logs indicate a generally stable and high-throughput connection for the UE (RNTI 'ff04'), but they also reveal several recurring synchronization and resource allocation warnings that suggest potential underlying physical layer or configuration issues.

## 🟢 Strengths (Good Performance Indicators)

1. High Throughput: The data transfer rates are excellent.
   * MAC Throughput: The 'MAC: TX' and 'RX' bytes consistently increase over time (e.g., from 73094 to 73066533 TX, and 66724 to 1070950 RX).
   * LCID 4 Throughput: The primary data channel (LCID 4) shows massive, sustained data transfer (e.g., 1,093,596 bytes at Slot 760.0, 1,557,256 bytes at Slot 304.0).

2. Low Error Rates (BLER): For the majority of the session, the Block Error Rate (BLER) is extremely low, often reaching 0.000005 or 0.000015. This indicates that the physical layer is successfully decoding most transmitted data.

3. Stable RSRP: The Received Signal Strength Indicator (RSRP) remains strong and stable, fluctuating between -84 dB and -95 dB, which is typical for a good cellular connection.

4. Successful RA Procedure: The initial Random Access (RA) procedure for 'ff04' was successful, confirming initial synchronization.

## 🟡 Weaknesses and Areas of Concern (Potential Issues)

1. Timing Advance (TA) Warnings (Critical):
   * The most frequent and concerning warning is: 'Invalid timing advance offset for RNTI ff04'.
   * Impact: Timing Advance (TA) is crucial for synchronizing the UE's uplink transmission timing with the gNB. Repeated "Invalid timing advance offset" warnings suggest that the UE is either calculating its timing incorrectly or the gNB is receiving timing information that is inconsistent or invalid. This can lead to intermittent uplink transmission failures or reduced capacity.

2. BLER Spikes (Moderate):
   * While most BLER values are near zero, there are noticeable spikes, particularly in the later slots (e.g., 0.127075, 0.137425, 0.163065).
   * Impact: These spikes indicate periods where the link quality degrades, leading to a higher percentage of corrupted data blocks.

3. Resource Utilization Imbalance (Minor):
   * The logs show that 'LCID 2' consistently reports 'TX 0 RX 0 bytes'.
   * Impact: If LCID 2 is intended for a specific service or control channel, the fact that it is never used suggests either a configuration mismatch or that the service associated with this logical channel is inactive.

4. RA Procedure Failures (Transient):
   * The logs show multiple instances of the UE failing to schedule PUSCH (e.g., 'UE ff04: Detected UL Failure on PUSCH after 10 PUSCH DTX, stopping scheduling').
   * Impact: This indicates the UE is repeatedly failing to maintain uplink scheduling, which is a symptom of potential timing, power, or resource allocation issues.

...
```

Results & Analysis - 2

- LLM translated retrieval query built directly from logs is either incorrect or nonsensical.
- Evidence to support:
 - shared_translated_query.txt for du_mac_logs_3.txt is:
'invalid timing advance offset and ul failure on pusch after dtx' (incorrect!)
 - shared_translation_query.txt for harq_exhaustion_logs_new_trimmed.txt is:
'harq nack scheduling stall mcs invalid cqi feedback sinr dl ctrl dl data' (not a meaningful English phrase!)
- Explanation: The LLM cannot accurately determine the root cause event unless a more targeted prompt is given.

Results & Analysis - 3

- LLM extracted top key events are often inaccurate/unhelpful:

In
results/harq_exhaustion_logs_new_trimmed/mode13/mode13_key_events_wo_rag.txt, we have:

1. [MAC][0.404500] DL HARQ | RNTI 1 | PID 12 | NACK | Retx 0
 2. [MAC][0.405500] DL HARQ | RNTI 1 | PID 15 | NACK | Retx 1
 3. [MAC][0.407000] DL HARQ | RNTI 1 | PID 12 | NACK | Retx 1
 4. [MAC][0.410500] DL HARQ | RNTI 1 | PID 15 | NACK | Retx 3
 5. [MAC][0.404500] DL HARQ | RNTI 1 | PID 12 | NACK | Retx 0
- *** HARQ EXHAUSTED → scheduling stall

Explanation: These events are rare in the trimmed log files. The LLM probably does not focus on these fewer but regularly occurring events like HARQ NACKs. Also, it is not working with log templates like the dynamic miner approach which makes it hard to key events and instead focus on metric values.

With mode7 (Dynamic miner + No RAG), more relevant events show up:

In
results/harq_exhaustion_logs_new_trimmed/mode7/mode7_key_events_wo_rag.txt, we have:

1. <*> NR HARQ NACK <*> for RNTI=1 <*> *** HARQ EXHAUSTED → scheduling stall
2. [MAC <*> DL SCHED | Frame <*> Slot <*> | RNTI 1 | MCS <MCS_INVALID> | TBS 136 B | HARQ <*>
3. [PHY <*> CQI FEEDBACK | RNTI 1 | WB-CQI <CQI_INVALID> | MCS <MCS_INVALID> | RI 1
4. <*> NR HARQ NACK <*> for RNTI=1 <*>
5. [MAC <*> DL HARQ | RNTI 1 | PID <*> | <*> | Retx <*>

Results & Analysis - 4

- A targeted prompt built using extracted key events performs much better than RAG in doing RCA.
- Evidence to support:
 - All modes without RAG but with key event extraction and targeted prompt build a causal chain where general prompts fail to even with RAG.
 - Let us compare gemma4 outputs of Mode 7 (Dynamic Miner + no RAG) with Mode 2 (Baseline + Concat RAG) on du_mac_logs_3.txt:
- Explanation: RAG does not provide LLM with the information to do deep causal analysis. Only gives information on definitions of MAC entities.

Results & Analysis - 4 (continued)

Mode 2

```
1 |This analysis focuses on the provided MAC layer logs, interpreting the key performance indicators (KPIs) and error messages against the provided 3GPP context documentation.
2
3 ## 📶 Wireless Link Quality Analysis
4
5 Overall, the logs demonstrate a **highly stable and high-throughput connection** for an extended period, indicating that the basic radio link establishment and data transfer mechanisms are functioning correctly. However, the latter half of the logs show a
**clear degradation in link quality and increased error rates**, suggesting potential environmental or configuration issues that need addressing.
6
7 ### 🟢 Strengths (What is working well)
8
9 1. **Successful Connection and Mobility:** The UE successfully completes the Random Access (RA) procedure and maintains connectivity across many slots (from Slot 384.0 up to Slot 896.0 in the final segment).
10 2. **High Throughput:** The MAC layer statistics show consistently high data transfer rates (e.g., TX reaching 10-11 Mbps in the later slots, and RX consistently above 1.2 MB/s). This indicates the physical layer and MAC scheduling are effectively
    utilizing the allocated resources.
11 3. **Stable Signal Strength (RSRP):** The average RSRP remains strong, generally fluctuating between -85 dBm and -95 dBm. While -95 dBm is approaching the limit, the consistency suggests the UE is maintaining a stable path loss profile.
12 4. **Low Initial Error Rates:** For the first several hundred slots, the BLER (Block Error Rate) is extremely low (e.g., 0.00000 to 0.00001), indicating excellent initial link quality.
13
14 ### 🟡 Weaknesses and Areas of Concern (Degradation and Errors)
15
16 The primary issues are concentrated in the latter half of the log file, suggesting the link quality is deteriorating over time or under sustained load.
17
18 #### 1. Increasing Error Rates (BLER and DTX)
19 * **BLER Trend:** The BLER (Block Error Rate) shows a clear upward trend. It starts near zero (e.g., 0.00000) and gradually increases, reaching values like **0.16586** (Slot 384.0, final segment) and **0.13742** (Slot 384.0, final segment).
20 * **Interpretation:** A BLER of 0.1 to 0.2 is significantly higher than the ideal target (typically < 0.1, or even < 0.05 for high-reliability services). This indicates that a substantial percentage of transmitted data blocks are corrupted or lost,
    leading to retransmissions and reduced effective throughput.
21 * **dl_sch_errors / ul_sch_errors:** The number of errors in both DL and UL HARQ processes increases significantly (e.g., 'dl_sch_errors' reaching 1396, 'ul_sch_errors' reaching 100).
22 * **Interpretation:** This confirms the BLER trend. The system is struggling to reliably acknowledge and retransmit data, suggesting interference or fading is impacting the control channels (PDCCH/PUCCH).
23 * **pucch0_DTX:** This metric (missed PUCCH detections) also rises, indicating difficulty in reliably conveying ACK/NACK feedback, which is critical for HARQ retransmissions.
24
25 #### 2. Timing and Synchronization Warnings
26 * **Invalid timing advance offset for RNTI ffd4:** This warning appears repeatedly and is a critical indicator.
27 * **Interpretation:** Timing Advance (TA) is used to correct the timing offset between the UE and the gNB. Repeated "Invalid timing advance offset" warnings mean the UE is either unable to accurately measure its timing offset or the gNB is receiving
    timing reports that are inconsistent or outside acceptable bounds. This can severely limit the achievable UL data rate and stability.
28
29 #### 3. MAC Layer Anomalies
30 * **Received unknown MAC header (LCID = 0x32) / Received unknown MAC header (LCID = 0x31):** These warnings indicate that the UE is receiving MAC PDUs with Logical Channel IDs (LCIDs) that are not expected or configured.
31 * **Interpretation:** While the MAC layer is designed to discard unknown PDUs (as per the context documentation), frequent unknown headers can point to interference, neighboring cell leakage, or misconfiguration in the network stack.
32 * **residual UL MAC PDU in 117.7 with length < 0!, pdu_len -15738:** This is a severe internal processing error, suggesting a buffer overflow or data corruption issue within the MAC layer processing of the PDU.
33
34 ---
35
36 ## 🟡 Suggested Improvement Strategies
```

Results & Analysis - 4 (continued)

Mode 7

3. Causal Chain Analysis

The root cause (timing misalignment) initiates a cascade of failures that degrade the connection quality, leading to the observed instability and eventual shutdown.

Stage	Event/Error	Description	Impact on Connection Quality
Initial State	Successful RA/Connection	The UE successfully completes the Random Access procedure ('CBRA procedure succeeded!') and establishes initial synchronization.	High (Initial connection established).
Root Cause Trigger	Invalid timing advance offset for RNTI ffd4	The UE's timing mechanism begins to drift or miscalculate the necessary timing advance offset.	Critical: Data transmitted by the UE is physically misaligned in time, causing the gNB to receive corrupted or unusable signals.
Level 1 Failure (MAC/PHY)	residual UL MAC PDU	Because the timing is wrong, the received bits are corrupted. The MAC layer attempts to parse the corrupted data, resulting in a protocol parsing failure.	Severe: Data integrity is lost. The connection cannot reliably transfer large amounts of data.
Level 2 Failure (MAC/Protocol)	Received unknown MAC header	The corruption is so severe that the MAC layer cannot even identify the data format, treating the payload as unknown or malformed.	Severe: The protocol stack cannot process the incoming data stream, leading to discarded packets.
Level 3 Failure (Application/Scheduler)	UE ffd4: Detected UL Failure on PUSCH	The cumulative effect of timing errors, corrupted PDUs, and unknown headers means the UE's scheduler repeatedly fails to confirm successful data transmission (PUSCH).	Critical: The UE assumes the link is unusable and stops attempting to transmit data, effectively dropping the connection.
Final State	Caught SIGTERM, shutting down	The system detects the persistent failure (UL Failure) and terminates the connection/thread.	Failure: Connection loss.

Results & Analysis - 5

- RAG provides almost no improvement to key event extraction based methods.
- We do a comparison of causal chains obtained from Mode 4 (Static+no_rag) and Mode 5 (Static+rag_concat)
- RAG based on specs + docs offers no information to perform deep root cause analysis of logs.

Results & Analysis - 5 (continued)

Mode 4

🔗 Causal Chain

The root cause ('Invalid timing advance offset') initiates a cascade of physical and logical errors, severely degrading the connection quality over time.

Phase 1: Physical Layer Degradation (The Root Cause)

- * ****Root Cause:**** 'Invalid timing advance offset for RNTI ffd4'
- * ****Immediate Effect:**** The gNB cannot accurately time-align the incoming uplink signals from the UE.
- * ****Symptom 1:**** The UE's uplink transmissions (PUSCH/UL-SCH) are received with timing misalignment.

Phase 2: Synchronization and Measurement Errors

- * ****Impact:**** Because the timing is off, the physical layer processing struggles to correctly decode the signal parameters.
- * ****Symptom 2:**** 'UE ffd4: reported RSRP index <*> invalid' (The UE is reporting invalid measurements, likely due to the timing issue corrupting the measurement data).
- * ****Symptom 3:**** 'Received unknown MAC header (LCID = <*>)' (The timing misalignment and corrupted physical layer data can cause the MAC layer to misinterpret the start or structure of the incoming packet, leading to unknown headers).

Phase 3: Connection Quality Degradation (The Observable Impact)

- * ****Impact:**** The cumulative effect of timing errors and corrupted packets leads to increased error rates and reduced throughput.
- * ****Metric Degradation:****
 - * ****BLER (Block Error Rate):**** The BLER values show a clear trend of increasing instability, especially in the later slots (e.g., moving from \$0.00000\$ to \$0.12974\$ and \$0.14367\$). This indicates that a significant portion of the transmitted data is corrupted or lost.
 - * ****UL-RI (Uplink Received Interference):**** While the value remains 1, the associated error rates confirm the instability.
 - * ****MAC/LCID Data:**** The overall throughput (TX/RX bytes) remains high, suggesting the connection is *trying* to maintain service, but the increasing BLER shows that the quality of the data transfer is rapidly deteriorating.

Phase 4: System Shutdown (The Final Outcome)

- * ****Impact:**** The persistent, unresolvable physical layer errors eventually trigger higher-level failure mechanisms.
- * ****Symptom:**** The logs show the UE detecting UL failures and eventually the system receiving a 'SIGTERM' signal, leading to the graceful shutdown of the radio processing threads ('[PHY] Stopping RU 0 processing threads', 'Bye.').

Results & Analysis - 5 (continued)

Mode 5

Causal Chain

1. Root Cause (PHY Layer Failure):

- * **Event:** Invalid timing advance offset for RNTI ffd4
- * **Mechanism:** The UE (ffd4) is unable to maintain or report a valid timing advance offset to the gNB.
- * **Immediate Effect:** The gNB's ability to correctly decode the timing of the UE's uplink transmissions (PUSCH) is compromised.

2. Secondary Impact (MAC/RLC Layer Degradation):

- * **Observation:** The logs show repeated instances of this error occurring across multiple slots (e.g., Slot 128.0, 256.0, 384.0, etc.).
- * **Effect:** The MAC layer, while attempting to process the PUSCH data, is constantly receiving timing warnings. Although the log shows the MAC layer *receiving* the data (e.g., 'UE ffd4: MAC: TX 76521083 RX 1918699 bytes'), the underlying timing instability means that the data integrity is questionable, and the system is operating under stress.

3. Tertiary Impact (Performance Metrics Deterioration):

- * **Observation:** The logs show the BLER (Block Error Rate) fluctuating and increasing significantly in the later slots:
 - * *Early Slots:* BLER approx 0.00000 (Excellent)
 - * *Mid Slots:* BLER approx 0.12 to 0.20 (Poor)
 - * *Late Slots:* BLER remains high (e.g., 0.14367, 0.16916, 0.11688).
- * **Mechanism:** The timing instability (Root Cause) leads to increased packet loss and decoding errors, which directly manifests as a high and unstable **BLER**.
- * **Observation:** The 'dlsch_errors' (DL SCH errors) and 'ulsch_errors' (UL SCH errors) also increase dramatically, indicating that the MAC layer is struggling to maintain reliable data transfer.

4. System Response (Mitigation/Failure):

- * **Observation:** The logs show the system attempting to recover or re-initialize the connection (e.g., the RA procedure for 'ad5d' and the subsequent 'Remove NR rnti 0xffd4').
- * **Effect:** The repeated failures force the system to perform complex recovery procedures (like re-running RA or removing the UE context), which consume resources and indicate a persistent, unresolved connection quality issue.

Limitations / Failure Cases

- Where does system break?
 - RAG on 3GPP specs + OAI docs. Good for Q&A purposes (as done in TelecomRAG), but not for RCA.
- Unrealistic assumptions (if any)
 - Sentence embedding and clustering assumes that anomalies are rare. If a specific MAC failure triggers cascading logs, then the resulting cluster will be large.
- Scalability issues?
 - N/A
- Overhead?
 - N/A

Comparison with Objectives

Show evidence supporting how each objective is achieved

Objective 1

Status (Completed/Partially Completed, Infeasible, etc)

- Replicate prompt dependency finding of LLM5G-MAC.
- Propose new scenarios causing MAC layer faults and collect relevant logs using OAI/ns3.
- Evaluate whether LLMs are effective at providing detailed analyses of such faults

Completed in mid-term presentation.

- Evaluate whether RAG assists in eliminating prompt dependency

Completed.

It does not if done on 3GPP specs and OAI docs.
Evidence: All artifacts corresponding to modes 2,3,5,6,8,9,11,12

- Explore alternative data-driven approaches to mitigate prompt dependency and close the operator skill gap.

Completed.

Evidence: All artifacts corresponding to modes 4,5,6,7,8,9,10,11,12,13

Comparison with Objectives

Show evidence supporting how each objective is achieved

Objective 1

- Compare and contrast the different approaches and assess the efficacy of each one.

Status (Completed/Partially Completed, Infeasible, etc)

Completed.
Executed a 13-mode comparative pipeline showing that RAG is not useful for RCA and that event extraction and targeted prompting is far more effective.

Evidence:

All the code files in the *scripts/* and *src/* directories.

scripts/full_pipeline.py,

scripts/mode13_pipeline.py,

src/mine_logs.py, etc.

The outputs in the *results/* directory.



Live Demo

- Scenario: Briefly describe what you are about to show in the live demo.
 - Will run the 13-mode pipeline and show all produced artifacts.
- (Switch to Live Demo / Show Backup Video here)

Work Distribution

Name	Core Responsibility	Key Tasks & Contributions
Dhanraj Shrivastav	Full Project	All tasks carried out individually

Conclusions

- Summary of key contributions
 - Key/anomalous event extraction mechanisms for logs that are used to build a targeted prompt for causal chain analysis.
 - The approaches successfully identify such events and produce precise causal chains.
 - Classical NLP and Embedding based methods (Static, dynamic miner, sentence clustering), One LLM based method.
- Impact of the work
 - All approaches successfully mitigate the issue of prompt dependency with the use of RAG or fine-tuning.

Conclusions (continued)

- Future Work

- If you are given 3 months in the summer, what would you add?
 - LLM based key event extraction was weak. Efforts can be dedicated to developing a specialized event extraction framework. Instruction fine tuning can be done here. Will require at least 1000 log files over varied scenarios with corresponding anomalies identified (ground truth). This is a data collection exercise. Feasible on a testbed.
- What can be extended to publication/deployment?
 - The above dataset can be released. This could be a publication.

References

- GitHub Repo Link:
<https://github.com/dhanraj-s/TWiN-IITH-5GMACTLogAnalysis>
 - Clearly mark code snippets generated using LLMs with comment "<Name of LLM> Generated Code Snippet for TWiN Project"
- References:
 - 1. *LLM-5GMACT: Performance Optimization in O-RAN Split 7.2 Using LLM-Based MAC-Layer Log Analysis* (<https://dl.acm.org/doi/10.1145/3737900.3770164>)
 - 2. *TelecomRAG: Taming Telecom Standards with Retrieval Augmented Generation and LLMs* (<https://dl.acm.org/doi/10.1145/3711992.3711996>)

Declaration on LLM Usage

- List which LLMs you leveraged
 - Gemini, Claude
- Include some sample prompts you used
 - “Here are the individual scripts with hardcoded paths and parameters. Create a single pipeline script that makes use of config.json to load the parameters and paths.”
 - “Here is the script. Now make it more robust by adding caching so that if the groq responses are already present, there is no need to call the API again.”
 - “What is the chi2 test?”
- How did you validate LLM generated scripts?
 - By going through all generated code carefully.
- How did you document your LLM generated code snippets?
 - Through comments, doc-strings, etc.